

Opening remarks

What the application does

Course discovery

Evaluations and attribution

Profiles, memories, and history

Schedules, calendar, and audits

Chat and natural-language edits

How the application works

Logical layers

Components and integrations

Deployment view

Chat request and SSE

Unified search pipeline

Mutations and clarification

Parallel audit and quality gate

Output validation (civility / policy)

Agent Overview

Automated tests, evaluation, and observability

Closing remarks

Limitations and future work

Questions

Atlas

AI-Assisted Schedule Builder for JHU Undergraduates

Alina Pan Jennifer He James Guo Junzhe Shi
Rachael Pei

Johns Hopkins University
Final presentation

May 4, 2026

One-liner: Natural-language course discovery, personalized schedules, and agentic planning—not a static catalog search.

Opening: Problem, Users, Why AI

Problem

JHU students need to discover courses, respect constraints, and build feasible schedules using SIS data and evaluations—without tab-hopping and guesswork.

Users

Undergraduates planning a term: search, shortlist, compare workload, and iterate with advising-style help.

Why AI is essential (not bolted-on)

- ▶ **Iter 1:** Semantic search + match explanations over embeddings; agent orchestrates tools.
- ▶ **Iter 2–3:** Onboarding → structured memories; schedule-aware chat; NL add/drop/swap.
- ▶ **Iter 4:** Clarification-gated mutations; unified search; parallel audits + quality gates.

Speaker: @Alinapanyue

Opening remarks

What the application does

Course discovery
Evaluations and attribution
Profiles, memories, and history
Schedules, calendar, and audits
Chat and natural-language edits

How the application works

Logical layers
Components and integrations
Deployment view
Chat request and SSE
Unified search pipeline
Mutations and clarification
Parallel audit and quality gate
Output validation (civility / policy)
Agent Overview
Automated tests, evaluation, and observability

Closing remarks

Limitations and future work
Questions

Demo 1 — Course discovery

Goal

Open-ended query → ranked results; unified search with `matchType`; optional disambiguation (Iter 4 R3).

- ▶ Show: natural language query → results list + explanations where semantic.
- ▶ Mention: `searchCourseDescriptions` (Iter 1) → orchestration (Iter 2 R6) → clarification state (Iter 4).

Speaker: @rachael-p Time: ~4 min

Opening remarks

What the application does

Course discovery

Evaluations and attribution

Profiles, memories, and history

Schedules, calendar, and audits

Chat and natural-language edits

How the application works

Logical layers

Components and integrations

Deployment view

Chat request and SSE

Unified search pipeline

Mutations and clarification

Parallel audit and quality gate

Output validation (civility / policy)

Agent Overview

Automated tests, evaluation, and observability

Closing remarks

Limitations and future work

Questions

Demo 2 — Eval summaries + attribution

Goal

On-demand eval summary grounded in metrics; raw/source table (Iter 4 R1).

- ▶ Show: “Summarize evals” on a course card → narrative + metrics.
- ▶ Open **raw** / **source** modal: terms, instructors, numeric rows aligned to claims.
- ▶ If no data: UI must not fabricate (Iter 1 R4).

Speaker: @JunzheShi0702 Time: ~4 min

Opening remarks

What the application does

Course discovery

Evaluations and attribution

Profiles, memories, and history

Schedules, calendar, and audits

Chat and natural-language edits

How the application works

Logical layers

Components and integrations

Deployment view

Chat request and SSE

Unified search pipeline

Mutations and clarification

Parallel audit and quality gate

Output validation (civility / policy)

Agent Overview

Automated tests, evaluation, and observability

Closing remarks

Limitations and future work

Questions

Demo 3 — Onboarding, memories, history

Goal

Preferences from onboarding + chat → long-term memories; completed courses / transcript path (Iter 4 R2).

- ▶ Show: onboarding or profile → `derived_memories` / memory list.
- ▶ Optional: completed-course history or transcript review flow (matched / ambiguous / unmatched).

Speaker: @James-Guo-03 Time: ~4 min

Opening remarks

What the application does

Course discovery

Evaluations and attribution

Profiles, memories, and history

Schedules, calendar, and audits

Chat and natural-language edits

How the application works

Logical layers

Components and integrations

Deployment view

Chat request and SSE

Unified search pipeline

Mutations and clarification

Parallel audit and quality gate

Output validation (civility / policy)

Agent Overview

Automated tests, evaluation, and observability

Closing remarks

Limitations and future work

Questions

Demo 4 — Schedules, calendar, audit

Goal

Dashboard → schedule page → workload audit; list vs. weekly calendar (Iter 4 R4).

- ▶ Run audit; show feasibility / workload narrative.
- ▶ Toggle **list** / **calendar**; mention parallel checks + synthesis (Iter 4 R5) at high level.

Speaker: @chjenniferhede Time: ~4 min

Opening remarks

What the application does

Course discovery

Evaluations and attribution

Profiles, memories, and history

Schedules, calendar, and audits

Chat and natural-language edits

How the application works

Logical layers

Components and integrations

Deployment view

Chat request and SSE

Unified search pipeline

Mutations and clarification

Parallel audit and quality gate

Output validation (civility / policy)

Agent Overview

Automated tests, evaluation, and observability

Closing remarks

Limitations and future work

Questions

Demo 5 — Schedule-aware chat + NL edits

Goal

Chat with schedule context; NL add/drop/swap via `modifyScheduleCourses`; clarification if ambiguous (Iter 3 R1, Iter 4 R3).

- ▶ One concrete NL command that mutates the schedule on deployed app.
- ▶ If time: trigger or describe clarification loop (saved state per conversation).

Speaker: @rachael-p (or handoff with @chjenniferhede for chat UI) Time: ~4 min

Opening remarks

What the application does

Course discovery

Evaluations and attribution

Profiles, memories, and history

Schedules, calendar, and audits

Chat and natural-language edits

How the application works

Logical layers

Components and integrations

Deployment view

Chat request and SSE

Unified search pipeline

Mutations and clarification

Parallel audit and quality gate

Output validation (civility / policy)

Agent Overview

Automated tests, evaluation, and observability

Closing remarks

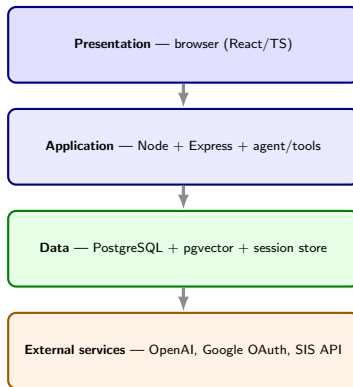
Limitations and future work

Questions

Stack: logical layers

Responsibilities

- ▶ **Client:** React + Vite, streaming UI, schedule views
- ▶ **API:** Express routes, auth session, agent endpoint
- ▶ **Data:** Postgres + pgvector, chat/audit persistence
- ▶ **External:** OpenAI, Google OAuth, JHU SIS



Speaker: @JunzheShi0702

Opening remarks

What the application does

Course discovery
Evaluations and attribution
Profiles, memories, and history
Schedules, calendar, and audits
Chat and natural-language edits

How the application works

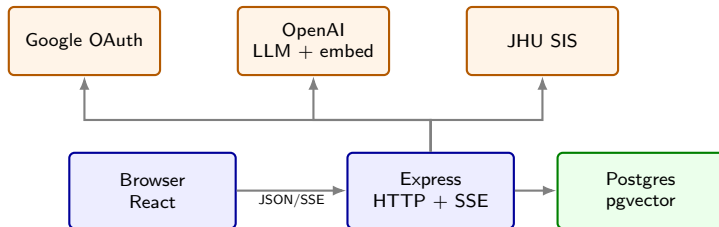
Logical layers

Components and integrations
Deployment view
Chat request and SSE
Unified search pipeline
Mutations and clarification
Parallel audit and quality gate
Output validation (civility / policy)
Agent Overview
Automated tests, evaluation, and observability

Closing remarks

Limitations and future work
Questions

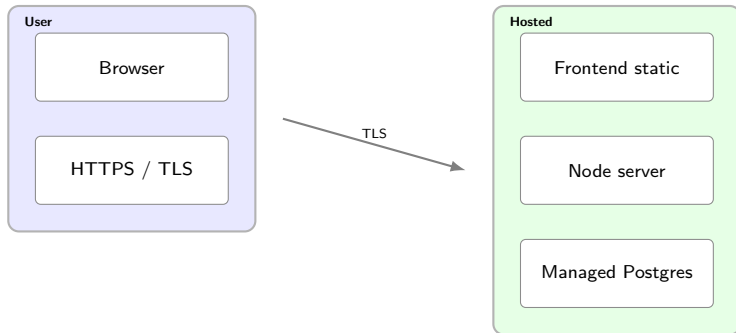
Stack: components (bird's-eye)



Integration notes

Session cookie authenticates dashboard routes; `/api/agent` upgrades to SSE for streaming tokens and tool progress (Iter 3 R6).

Deployment (typical)



Swap placeholder host names on the right for your real stack (e.g. Vercel + Railway + Supabase).

Opening remarks

What the application does

- Course discovery
- Evaluations and attribution
- Profiles, memories, and history
- Schedules, calendar, and audits
- Chat and natural-language edits

How the application works

- Logical layers
- Components and integrations

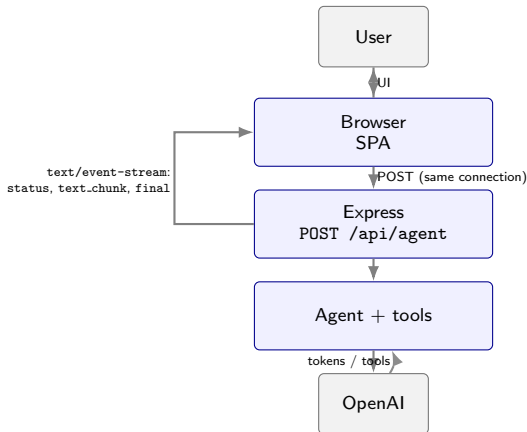
Deployment view

- Chat request and SSE
- Unified search pipeline
- Mutations and clarification
- Parallel audit and quality gate
- Output validation (civility / policy)
- Agent Overview
- Automated tests, evaluation, and observability

Closing remarks

- Limitations and future work
- Questions

Runtime: chat request and SSE



Persistence (Iter 3)

User message saved immediately; assistant text on final; partial + aborted metadata on stop.

Speakers: @rachael-p, @Alinapanyue

Opening remarks

What the application does

- Course discovery
- Evaluations and attribution
- Profiles, memories, and history
- Schedules, calendar, and audits
- Chat and natural-language edits

How the application works

- Logical layers
- Components and integrations
- Deployment view

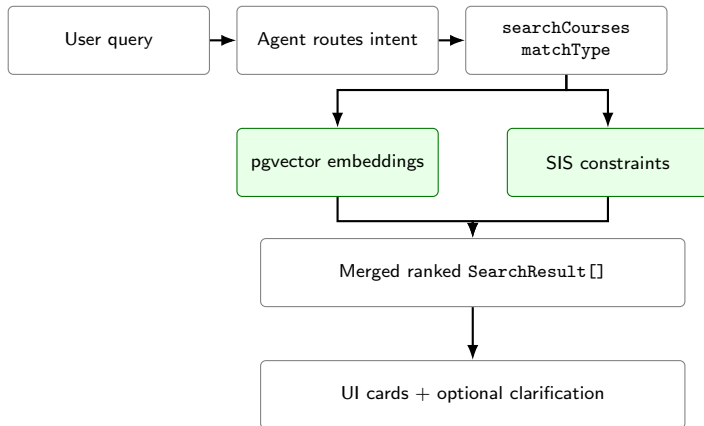
Chat request and SSE

- Unified search pipeline
- Mutations and clarification
- Parallel audit and quality gate
- Output validation (civility / policy)
- Agent Overview
- Automated tests, evaluation, and observability

Closing remarks

- Limitations and future work
- Questions

Data flow: unified course search



Iter 4 R3: one merged search surface; semantic vs. exact vs. hybrid encoded per row.

Opening remarks

What the application does

- Course discovery
- Evaluations and attribution
- Profiles, memories, and history
- Schedules, calendar, and audits
- Chat and natural-language edits

How the application works

- Logical layers
- Components and integrations
- Deployment view
- Chat request and SSE

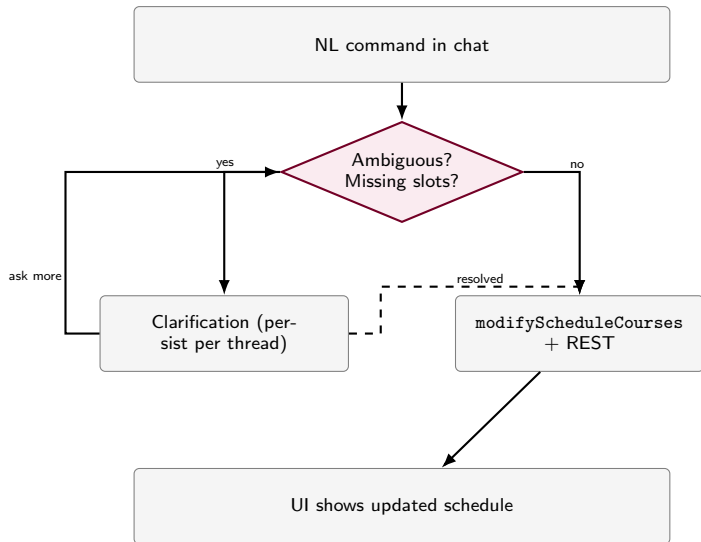
Unified search pipeline

- Mutations and clarification
- Parallel audit and quality gate
- Output validation (civility / policy)
- Agent Overview
- Automated tests, evaluation, and observability

Closing remarks

- Limitations and future work
- Questions

Control flow: NL schedule edit



Iter 3 R1 + Iter 4 R3: block mutations until required clarification slots are confirmed.

Opening remarks

What the application does

- Course discovery
- Evaluations and attribution
- Profiles, memories, and history
- Schedules, calendar, and audits
- Chat and natural-language edits

How the application works

- Logical layers
- Components and integrations
- Deployment view
- Chat request and SSE
- Unified search pipeline

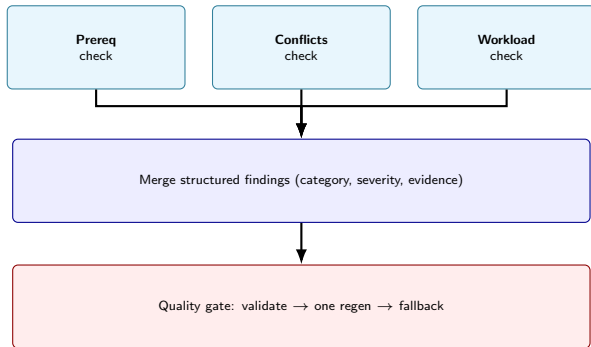
Mutations and clarification

- Parallel audit and quality gate
- Output validation (civility / policy)
- Agent Overview
- Automated tests, evaluation, and observability

Closing remarks

- Limitations and future work
- Questions

Parallel audit synthesis and gate



Failure isolation (Iter 4 R5)

One failed check still returns partial findings; gate telemetry (`gate_pass`, `gate_fail`, ...) where implemented.

Opening remarks

What the application does

- Course discovery
- Evaluations and attribution
- Profiles, memories, and history
- Schedules, calendar, and audits
- Chat and natural-language edits

How the application works

- Logical layers
- Components and integrations
- Deployment view
- Chat request and SSE
- Unified search pipeline
- Mutations and clarification
- Parallel audit and quality gate**
- Output validation (civility / policy)
- Agent Overview
- Automated tests, evaluation, and observability

Closing remarks

- Limitations and future work
- Questions

Opening remarks

What the application does

- Course discovery
- Evaluations and attribution
- Profiles, memories, and history
- Schedules, calendar, and audits
- Chat and natural-language edits

How the application works

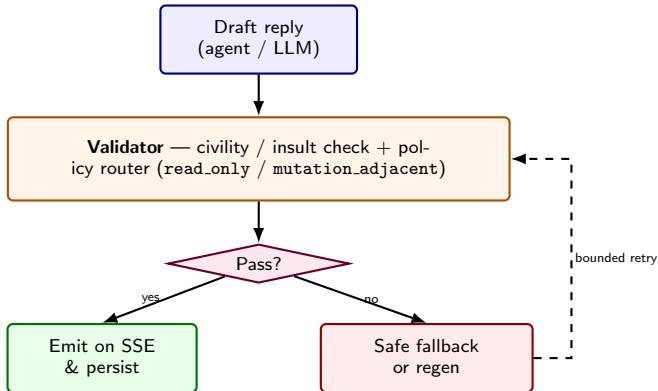
- Logical layers
- Components and integrations
- Deployment view
- Chat request and SSE
- Unified search pipeline
- Mutations and clarification
- Parallel audit and quality gate
- Output validation (civility / policy)**
- Agent Overview
- Automated tests, evaluation, and observability

Closing remarks

- Limitations and future work
- Questions

User-visible text: validator before stream (Iter 4)

Assistant text can occasionally include **insults** or other **disallowed** content. A **validator** runs on draft output *before* chunks are committed to the SSE stream / persisted, so users never see toxic lines when the gate fires.



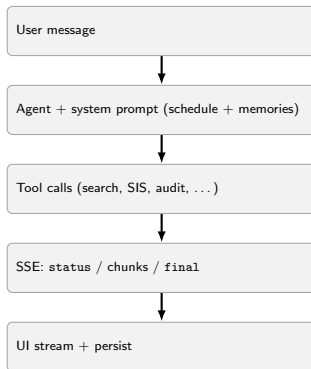
Same family as quality gates (Iter 4 R5)

Treat civility like other **hard failures**: validate → at most one constrained regen → **safe fallback** if still failing; log `gate_fail` / policy reason for observability.

Agent loop (summary)

Tooling (Iter 1–4)

- ▶ Models: e.g. gpt-4o-mini; text-embedding-3-small
- ▶ Search, SIS details, eval summary, metrics, audits, schedule mutate
- ▶ Unified searchCourses; clarification state before writes



Opening remarks

What the application does

Course discovery
Evaluations and attribution
Profiles, memories, and history
Schedules, calendar, and audits
Chat and natural-language edits

How the application works

Logical layers
Components and integrations
Deployment view
Chat request and SSE
Unified search pipeline
Mutations and clarification
Parallel audit and quality gate
Output validation (civility / policy)

Agent Overview

Automated tests, evaluation, and observability

Closing remarks

Limitations and future work
Questions

Testing, AI eval, observability

Non-AI

Vitest (backend + frontend); Playwright e2e; CI integration (final delivery rubric).

AI evaluation

Golden prompts / structured checks; agent regression paths; gate telemetry where shipped (Iter 4 R5).

Coverage screenshot

Add `coverage.png` next to `slides.tex` to show coverage here.
(@James-Guo-03 + @chjenniferhede)

Opening remarks

What the application does

- Course discovery
- Evaluations and attribution
- Profiles, memories, and history
- Schedules, calendar, and audits
- Chat and natural-language edits

How the application works

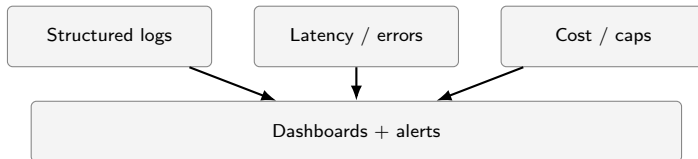
- Logical layers
- Components and integrations
- Deployment view
- Chat request and SSE
- Unified search pipeline
- Mutations and clarification
- Parallel audit and quality gate
- Output validation (civility / policy)
- Agent Overview

Automated tests, evaluation, and observability

Closing remarks

- Limitations and future work
- Questions

Ops feedback loop (conceptual)



Final delivery rubric: tie spoken claims to what is actually deployed (logging, rate limits, validation). Mark items as **shipped** vs. **planned** live.

Opening remarks

What the application does

- Course discovery
- Evaluations and attribution
- Profiles, memories, and history
- Schedules, calendar, and audits
- Chat and natural-language edits

How the application works

- Logical layers
- Components and integrations
- Deployment view
- Chat request and SSE
- Unified search pipeline
- Mutations and clarification
- Parallel audit and quality gate
- Output validation (civility / policy)
- Agent Overview

Automated tests, evaluation, and observability

Closing remarks

- Limitations and future work
- Questions

Closing: Limitations, future work, reflection

Limitations (be honest)

- ▶ Partial eval coverage; SIS rate limits / stale sections; edge cases in clarification and gates.
- ▶ Model failures: fallbacks and user-visible errors still evolving.

Future work

Stronger observability dashboards; broader course history; tighter prompt/tool contracts; more automated AI evals.

Reflection

Team coordination across four iterations; using AI tools as dev partners; trade-offs (latency vs. depth, autonomy vs. safety).

Speaker: @James-Guo-03 (different from opening)

Opening remarks

What the application does

Course discovery

Evaluations and attribution

Profiles, memories, and history

Schedules, calendar, and audits

Chat and natural-language edits

How the application works

Logical layers

Components and integrations

Deployment view

Chat request and SSE

Unified search pipeline

Mutations and clarification

Parallel audit and quality gate

Output validation (civility / policy)

Agent Overview

Automated tests, evaluation, and observability

Closing remarks

Limitations and future work

Questions

Thank you — Questions

Questions?

Be ready to **live-demo** any requirement on request (rubric).

Reference: docs/iteration-4-plan.md (clarification, gates, unified search).

Opening remarks

What the application does

- Course discovery
- Evaluations and attribution
- Profiles, memories, and history
- Schedules, calendar, and audits
- Chat and natural-language edits

How the application works

- Logical layers
- Components and integrations
- Deployment view
- Chat request and SSE
- Unified search pipeline
- Mutations and clarification
- Parallel audit and quality gate
- Output validation (civility / policy)
- Agent Overview
- Automated tests, evaluation, and observability

Closing remarks

Limitations and future work

Questions